아이디어 보조 어플 시스템 아키텍처 설계서 (통합본)

Oct 14, 2026 · @민성

React Native 앱 → Node.js API 서버(익명 토큰 발급 · 조합 조회) → PostgreSQL+pgvector → 같은 조합이 DB에 없을 때만 외부 LLM(Gemini). 사진과 로그인은 쓰지 않습니다.

STEP 1. 선택 지점 결정

항목	내용
팀명	5조 (기획서는 4조로 적혀 있어 팀 확인 필요)
아키텍처의 A	지태웅 (기획서의 A와 같은 사람으로 가정, 팀 확인 필요)

선택 지점	우리 팀의 선택	이유 (팀에 할 줄 아는 사람, 비기능 요구사항, 배포)	버린 선택지
클라이언트 기술	React Native (Expo) + TypeScript, 모바일 앱, Android 우선. 기기 저장은 SQLite	기획서에서 '모바일 앱'으로 정했고, 성능 목표(리롤 0.5초 · AI 5초)를 실기기로 재기로 함. 한 코드로 Android·iOS를 함께 만들 수 있음. 팀이 React 기반으로 정했고, 서버도 TypeScript라 언어를 하나로 맞출 수 있음. React·JavaScript를 다룰 수 있는 팀원은 확인 필요	Flutter(Dart), Kotlin·Swift 각각 개발(두 벌), React 웹 앱(PWA)

선택 지점	우리 팀의 선택	이유 (팀에 할 줄 아는 사람, 비기능 요구사항, 배포)	버린 선택지
서버 기술	Node.js (TypeScript) API 서버, Fastify. 서버 코드는 직접 짜고 실행 환경(PaaS)은 빌려 쓴다	LLM API 키를 앱에 두면 안 되므로(보안 요구사항) 서버 환경변수에만 보관. 'DB에 같은 조합이 있으면 꺼내고 없으면 AI 호출' 로직을 서버 한 곳에서 처리. 앱과 같은 TypeScript라 언어가 하나이고 요청·응답 타입을 공유할 수 있음. 서버가 상시 실행이라 콜드 스타트·실행 시간 제한이 없고, 대기 시간 대부분이 Gemini·DB 응답이라 Node의 비동기 처리에 맞음. 배포·토큰·호출 제한은 직접 만들어야 하며, 5초 목표는 무료 PaaS의 절전 후 첫 응답까지 포함해 9주차에 미리 측정	Supabase(Edge Functions + 인증), Spring Boot(앱과 언어가 다르고 메모리·시작 시간 부담), Firebase Cloud Functions, 앱에서 LLM 직접 호출(키 노출)
데이터베이스	PostgreSQL + pgvector (관리형 DB 서비스)	카테고리·단어·조합 결과·공유 글·반응이 서로 연결된 관계형 데이터. 같은 조합 1개만 저장(유일 키), 글마다 반응 1개 제한 같은 규칙을 DB가 보장. pgvector로 '연관 단어 찾기'를 같은 DB에서 처리. 카테고리당 100개 이상 단어에서 랜덤 3개는 쿼리 한 번이라 0.5초 안에 충분하다고 보고, 느리면 인덱스를 조정	Firestore(NoSQL), MongoDB, 앱 안 SQLite만 사용(다른 사용자와 공유 불가)

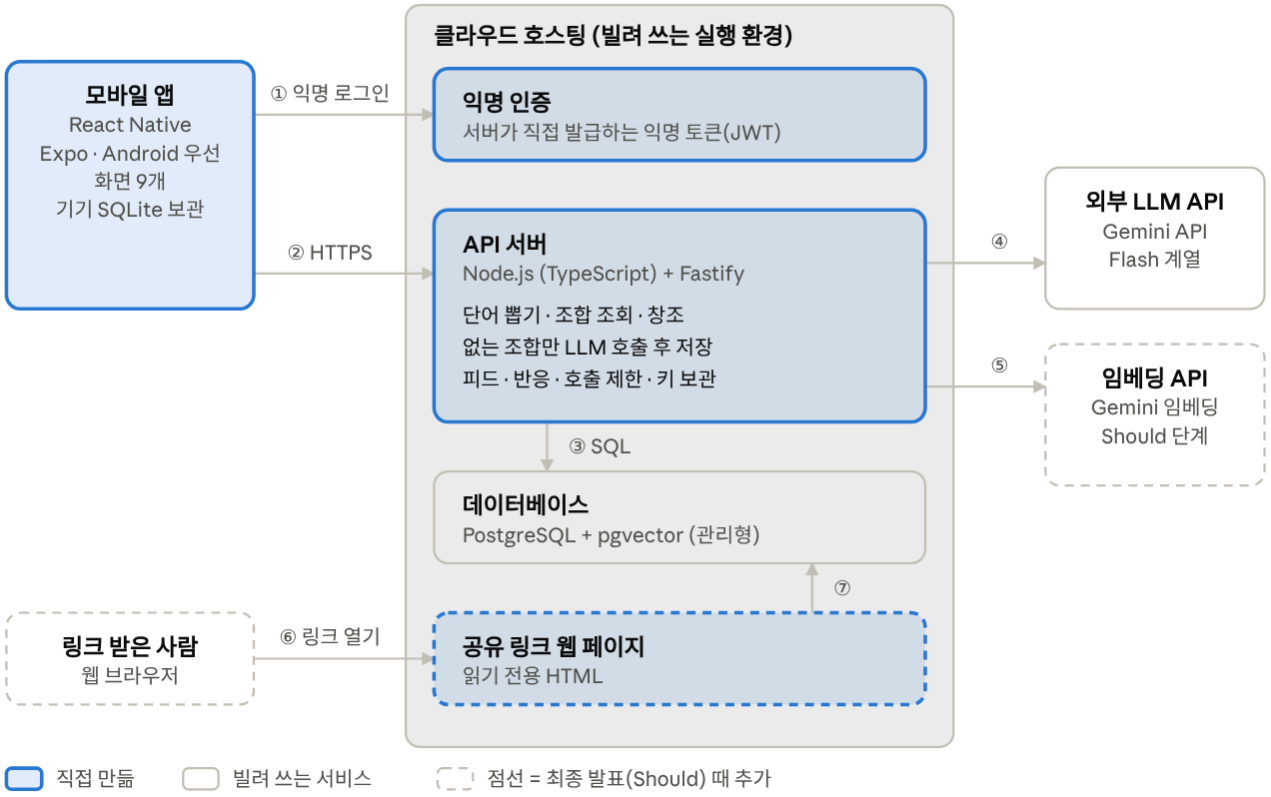
선택 지점	우리 팀의 선택	이유 (팀에 할 줄 아는 사람, 비기능 요구사항, 배포)	버린 선택지
사진 저장	사용하지 않음 (텍스트만)	기획서에서 '사진 사용 안 함'으로 결정, 화면 명세의 모든 화면에도 이미지 금지. 피드·공유 링크도 텍스트 카드로 충분하고 저장소 비용·부적절한 이미지 관리 부담이 없음	Supabase Storage·AWS S3에 사진 저장, AI 이미지 생성
인증	회원가입·로그인 없음. 첫 실행 때 서버가 기기마다 익명 ID를 만들고 서명한 토큰(JWT)을 발급	기획서 결정대로 로그인은 받지 않음. 다만 '내 글' 표시, 글마다 반응 1개, 기기별 AI 호출 횟수 제한에 기기 구분이 필요. 첫 실행 때 자동으로 토큰을 받고, 서버가 서명한 토큰이라 직접 검증할 수 있어 앱이 임의로 만든 ID보다 조작에 강함. 발급·검증 코드는 직접 만들어야 함(검증된 라이브러리 사용). 한계: 앱을 재설치하면 '내 글' 연결이 끊김	이메일·소셜 로그인, 아무 식별 없음(반응 중복·호출 남용을 막을 수 없음)
배포 위치	앱: Expo(EAS)로 빌드한 APK를 Google Play 내부 테스트로 배포(개발 중에는 Expo Go로 실기기 확인). API 서버(공유 링크 웹 페이지 포함): PaaS(Render·Railway 등), DB: 관리형 PostgreSQL. 모두 HTTPS	실기기 테스트·시연은 Android 내부 테스트로 충분(iOS는 시간이 되면). 서버와 DB는 가까운 리전(서울이 없으면 도쿄·싱가포르 등)에 두어야 0.5초·5초 목표에 유리하며, 리전은 가입 때 확인. 무료 플랜은 일정 시간 쓰지 않으면 서버가 절전하거나 DB가 멈출 수 있어 시연·성능 측정 전에 유료 최저 플랜 여부를 확인	AWS EC2 직접 배포, 학교 서버, 정식 앱 스토어 출시

선택 지점	우리 팀의 선택	이유 (팀에 할 줄 아는 사람, 비기능 요구사항, 배포)	버린 선택지
(팀이 추가) AI 호출	외부 LLM API: Google Gemini API의 Flash 계열 소형 모델. 같은 조합 키가 DB에 없을 때만 호출	결과 5초 이내 목표라 빠르고 저렴한 소형 모델. 프롬프트를 '카테고리 + 세 단어(+ 방향성)' 형식으로 고정해 진부한 답을 줄임. 만든 결과는 DB에 저장해 다음부터 호출 0회. 모델 버전·무료 사용량·호출 한도는 9주차 연동 때 확인. 임베딩도 같은 Gemini API를 쓰면 키와 청구가 한 곳	매번 AI 호출, 자체 AI 모델 개발(기획서 Won't), 다른 LLM 제공사(Claude 등), 큰 모델(느리고 비쌈)
(팀이 추가) 연관 단어 매칭 (Should)	임베딩 벡터 유사도: Gemini 임베딩 모델로 벡터를 만들어 pgvector에 저장	나만의 카테고리·방향성 입력은 미리 준비할 수 없어 의미가 가까운 단어를 자동으로 찾아야 함. 단어 벡터는 한 번만 계산해 저장하므로 뽑을 때는 DB 쿼리만 사용(리롤 0.5초 유지). 벡터 차원은 고른 임베딩 모델에 맞춰 DB 컬럼을 만들어야 함	카테고리 추가 때마다 LLM이 단어 고르기(느리고 비쌈), 단어에 태그를 손으로 달기

STEP 2. 구성도

상자 7개와 화살표 7개입니다. 앱은 우리 API 서버하고만 통신하고, LLM·임베딩 호출과 API 키는 서버 안쪽에서만 일어납니다.

앱은 우리 API 서버하고만 통신하고, 외부 AI는 서버만 부른다



시스템 구성도 · 상자 7개, 화살표 7개

굵은 파란 상자가 우리가 직접 만드는 부분이고, 회색 배경은 서버를 올려 두는 클라우드 호스팅 영역이며, DB는 그 안의 관리형 서비스입니다. 화살표 번호는 아래 화살표 목록과 같습니다.

상자 목록

상자	무엇인가	기술	우리가 만드나 / 빌려 쓰나
모바일 앱	사용자가 보는 화면 9개 (카테고리 선택, 나만의 카테고리 추가, 랜덤 단어, 결과 준비, 결과, 아이디어 확정, 만드는 법, 공유 시트, 공유 피드). 저장한 아이디어를 기기에 보관	React Native (Expo) + TypeScript, 기기 저장은 SQLite	만든다
API 서버	단어 뽑기, 조합 캐시 조회, AI 호출, 카테고리 매칭, 피드·반응 처리. 익명 토큰 발급·검증, 기기별 호출 횟수 제한, API 키 보관	Node.js (TypeScript) + Fastify	만든다 (실행 환경은 빌려 쓴다)

상자	무엇인가	기술	우리가 만드나 / 빌려 쓰나
데이터베이스	카테고리, 단어, 조합 결과, 만드는 법, 공유 글, 반응, 익명 사용자, 신고 저장. 연관 단어 검색	PostgreSQL + pgvector (관리형 DB)	빌려 쓴다 (테이블·쿼리·단어 입력은 우리가)
익명 인증	로그인 없이 기기마다 익명 ID·토큰 발급	서버가 직접 발급·검증하는 익명 토큰(JWT)	만든다 (API 서버에 포함)
외부 LLM API	세 단어 조합 아이디어(제목+설명), 만드는 법(재료·순서) 생성	Google Gemini API (Flash 계열 소형 모델)	빌려 쓴다 (프롬프트만 우리가 작성)
임베딩 API (Should)	단어·카테고리 이름·방향성 문장을 벡터로 바꾸어 연관 단어 찾기	Google Gemini API (임베딩 모델)	빌려 쓴다
공유 링크 웹 페이지 (Should)	앱이 없는 사람이 링크로 아이디어를 읽기 전용으로 봄	API 서버가 HTML 반환	만든다 (API 서버에 포함)

화살표 목록

어디서	어디로	무엇을 주고받나
① 모바일 앱	익명 인증	첫 실행 때 기기 값을 보내 익명 로그인 요청 → 익명 사용자 ID·토큰(JWT)을 받음
② 모바일 앱	API 서버	요청 + 토큰(카테고리 목록, 단어 뽑기, 창조, 만드는 법, 카테고리 추가, 피드 글·반응) → JSON 응답. 모두 HTTPS
③ API 서버	데이터베이스	랜덤 단어 추출·연관 단어 검색, 조합 키로 캐시 조회·저장, 피드·반응 읽기·쓰기(SQL) → 조회 결과
④ API 서버	외부 LLM API	DB에 같은 조합이 없을 때만 카테고리 + 세 단어(+ 방향성) 프롬프트 → 아이디어 제목·내용 또는 만드는 법 텍스트. 키는 서버 환경변수에만 둔다
⑤ API 서버	임베딩 API (Should)	새 카테고리 이름·방향성 문장 → 벡터(DB에서 가까운 단어를 찾는 데 사용). 같은 문장은 저장해 두고 다시 부르지 않는다

어디서	어디로	무엇을 주고받나
⑥ 링크 받은 사람 (브라우저)	공유 링크 웹 페이지 (Should)	공유 링크 열기 → 읽기 전용 아이디어 페이지 (HTML). 로그인 없음
⑦ 공유 링크 웹 페이지 (Should)	데이터베이스	공유 코드로 글 조회 → 아이디어 제목·내용·사용한 단어

STEP 3. 데이터 목록

서버 DB에 9덩어리, 기기에 1덩어리를 둡니다. 핵심은 '조합 결과'의 조합 키(카테고리 + 정렬한 단어 3개 + 방향성)이며, 이 키가 같은 조합의 AI 호출 0회를 보장합니다.

덩어리	들어가는 정보	어느 화면에서 나왔나	다른 덩어리와의 관계
카테고리	ID, 이름(2~20자, 중복 불가), 종류(기본 / 사용자 추가), 이름 벡터, 선택된 횟수('많이 쓴 순' 정렬용), 만든 익명 ID(비공개), 만든 시각	01 카테고리 선택, 01-1 나만의 카테고리 추가, 02 랜덤 단어	카테고리-단어 매칭을 통해 단어와 N:M. 조합 결과가 참조
단어	ID, 단어 글자, 임베딩 벡터(Should)	02 랜덤 단어 3개, 01-1 연관 단어 미리보기 (관리자가 미리 입력, 기본 카테고리당 100개 이상)	매칭을 통해 카테고리와 N:M. 조합 결과 하나에 3개씩 쓰임
카테고리-단어 매칭	카테고리 ID, 단어 ID, 연관 점수	01-1 (연관 단어 매칭 결과), 02 (뽑기 대상)	카테고리 1:N, 단어 1:N. 기본 카테고리는 미리 입력, 나만의 카테고리는 매칭된 단어(10개 이상)를 저장
조합 결과 (아이디어)	ID, 카테고리 ID, 단어 3개 ID, 방향성 문장(공백 정리한 값, 없으면 빈 값), 조합 키(유일), 아이디어 제목·내용, 생성 시각, 신고 수, 숨김 여부	02 '창조' → 03 결과 준비, 04 결과, 05 아이디어 확정	카테고리 N:1, 단어 3개 참조, 만드는 법 1:1, 공유 글 1:N. 조합 키는 중복 불가 (unique)
만드는 법	조합 결과 ID, 요약(걸리는 시간·난이도·분량), 재료 목록, 만드는 순서 4~6단계	05-1 만드는 법	조합 결과 1:1 (한 번 만들면 재사용). 음식 외 카테고리는 '실행 단계'로 저장

덩어리	들어가는 정보	어느 화면에서 나왔나	다른 덩어리와의 관계
공유 글	ID, 조합 결과 ID, 공개 범위(링크만 / 피드), 익명 ID, 익명 닉네임, 공유 링크 코드(추측 불가 값), 올린 시각, 반응 수	06 공유 시트, 07 공유 피드, 공유 링크 웹 페이지	조합 결과 N:1 (카드의 제목·내용·단어는 여기서 읽음), 반응 1:N
반응	공유 글 ID, 익명 ID, 종류(좋아요 / 그저그래요 / 별로에요), 시각	07 공유 피드	공유 글 N:1. (글, 익명 ID) 한 쌍당 1개만 있고, 취소하면 삭제
익명 사용자	익명 ID, 닉네임(#번호), 처음 실행 시각	앱 처음 실행 (모든 화면 뒤에서)	공유 글·반응·사용자 추가 카테고리의 주인. 이름·연락처는 받지 않음
신고 (화면 미설계)	대상(조합 결과 또는 공유 글) ID, 익명 ID, 시각	위험 2·4 대응으로 추가 (화면 명세에는 없음)	조합 결과·공유 글 N:1. 기기당 대상 1회만. 일정 수 이상이면 숨김
기기에 저장한 아이디어 (서버 DB 아님)	조합 결과 ID, 제목, 설명, 사용한 단어 3개, 만드는 법(있으면), 저장 시각	05 '저장'(Should), 기록 모아보기(Could)	서버에 올리지 않음. 조합 결과 ID로 나중에 공유할 때 서버 데이터와 연결

STEP 4. API 목록

핵심 흐름 9개를 먼저 적었습니다. 중간 발표(Must)에 필요한 것은 처음 4개이고, 나머지는 최종 발표(Should) 범위입니다. 익명 로그인을 뺀 모든 요청은 HTTPS에 익명 토큰을 붙이고, 서버는 먼저 토큰이 유효한지 확인합니다. 토큰은 서버가 직접 발급합니다.

화면의 행동	요청 이름	서버가 확인할 것	돌려주는 것
00 앱 처음 실행 (Must)	POST /auth/anonymous {기기 값}	기기 값 형식이 맞는지 / 같은 기기 값이면 기존 익명 ID를 다시 사용 / 기기당 발급 횟수 제한	익명 ID, 닉네임, 토큰(JWT)과 만료 시각
01 화면 열기 (Must)	`GET /categories`	토큰 유효	기본 카테고리 + 다른 사용자가 만든 카테고리(많이 쓴 순). 없으면 빈 목록

화면의 행동	요청 이름	서버가 확인할 것	돌려주는 것
02 카테고리 누름, 다시 뽑기, 방향성 입력 (Must)	`POST /words/draw` {카테고리 ID, 고정 단어 ID[], 방향성}	카테고리가 있는지 / 고정 단어가 2개 이하이고 그 카테고리 소속인지 / 방향성 50자 이하(공백 정리) / 방향성이 있으면 맞는 단어가 3개 이상인지. 방향성 벡터는 처음 한 번만 계산해 저장해 두고 재사용	단어 3개(고정 단어 그대로 + 나머지 랜덤). 단어 부족·'방향을 넓혀 보세요' 코드. 목표 0.5초 이내
02→03 '창조' 누름 (Must)	`POST /ideas` {카테고리 ID, 단어 ID 3개, 방향성}	단어 3개가 서로 다르고 그 카테고리 소속인지 / 조합 키로 DB 먼저 조회, 있으면 AI 호출 0회 / 없으면 기기별 호출 횟수 제한 확인 후 LLM 호출(5초 제한), 결과 형식(제목+설명)이 맞을 때만 저장, 실패는 저장 안 함 / 같은 조합이 동시에 와도 유일 키로 한 건만 저장	아이디어 ID, 제목, 내용, 단어 3개, DB에서 꺼냈는지 여부(목표 4번 확인용). 실패는 오류 코드
05→05-1 '진짜 만드는 법' 누름 (Should)	`POST /ideas/{id}/recipe`	아이디어가 있는지 / 저장된 만드는 법이 있으면 그대로(AI 0회), 없을 때만 LLM 생성 후 저장 / 호출 횟수 제한	시간·난이도·분량, 재료, 만드는 순서 4~6단계. 만들 수 없으면 '만들기 어려운 아이디어' 코드

화면의 행동	요청 이름	서버가 확인할 것	돌려주는 것
01-1 '추가하고 시작' 누름 (Should)	`POST /categories` {이름}	2~20자·금칙어 / 같은 이름이 있으면 새로 만들지 않고 기존 ID 반환 / 연관 단어 10개 이상 / 기기당 하루 추가 횟수 제한	카테고리 ID(→ 02로 이동) 또는 '이미 있음'·'단어 부족'
06 '피드에 올리기' (Should)	`POST /posts` {아이디어 ID}	아이디어가 있고 숨김 처리되지 않았는지 / 같은 기기가 같은 아이디어를 두 번 올리지 않았는지 / 호출 횟수 제한	글 ID, 공유 링크 코드, 익명 닉네임
07 피드 열기, 최신순·인기순 바꾸기 (Should)	`GET /posts?sort=latest|popular&cursor=`	정렬 값이 맞는지 / 한 번에 20개 이하 / 숨김 글 제외	글 목록(닉네임, 카테고리, 시각, 제목, 내용, 단어, 반응 수, 내 반응, 내 글 여부) + 다음 커서. 없으면 빈 목록
07 반응 누르기·바꾸기·취소 (Should)	`PUT /posts/{id}/reaction` {종류 또는 취소}	내 글이 아닌지 / 종류가 3개 중 하나인지 / (글, 익명 ID)당 1개(다른 걸 누르면 덮어쓰기)	바뀐 반응 수 3개, 내 반응

다음에 추가할 API (오늘 범위 밖)

화면의 행동	요청 이름	서버가 확인할 것	돌려주는 것
01-1 이름 입력 중 미리보기 (Should)	`POST /categories/preview` {이름}	2~20자·금칙어, 호출 횟수 제한(임베딩 비용)	연관 단어 개수·예시, 중복 이름 여부. 10개 미만이면 추가 불가 표시
06 '링크 복사' (Should)	`POST /shares` {아이디어 ID}	아이디어가 있는지, 호출 횟수 제한	공유 링크 코드(피드에는 안 올라감, 공유 글의 공개 범위 = 링크만)

화면의 행동	요청 이름	서버가 확인할 것	돌려주는 것
공유 링크 열기, 웹 (Should)	`GET /s/{코드}`	코드가 있고 숭김 처리되지 않았는지. 로그인 없음	읽기 전용 HTML(제목·설명·사용한 단어, 만드는 법이 있으면 함께)
04 '새로 생성' (미정)	`POST /ideas/{id}/regenerate`	호출 횟수 제한	새 아이디어. 같은 조합에 추가 저장하려면 조합 키 유일 조건을 '조합 키 + 버전'으로 바꾸어야 함
신고 (화면 미설계, 위험 2·4 대응)	`POST /reports` {아이디어 ID 또는 글 ID}	같은 기기의 중복 신고	접수 여부. 일정 수 이상이면 숨김

STEP 5. 결정 기록 두 장

아키텍처에 가장 크게 영향을 주는 두 결정을 골랐습니다. 팀이 실제로 더 오래 토론한 결정이 따로 있으면 그걸로 바꿔 쓰면 됩니다.

결정 기록 1

항목	내용
결정 제목	서버 기술: Supabase를 쓰지 않고 Node.js 서버를 직접 만든다
상황	LLM API 키를 앱 밖(서버)에 숨겨야 하고, 같은 조합이면 DB에서 꺼내는 로직이 필요하다. 로그인은 없지만 피드 반응·호출 제한 때문에 기기 구분은 필요하다. 11주차 중간 발표까지 핵심 루프를 완성해야 하고, 서버 운영 경험은 많지 않다고 본다(팀 확인)
선택지	① Node.js(TypeScript) 서버 + 관리형 PostgreSQL ② Spring Boot 서버 + PostgreSQL ③ Supabase (PostgreSQL + Edge Functions + 익명 로그인) ④ Firebase (Firestore + Cloud Functions)
결정	① Node.js 서버 + 관리형 PostgreSQL

항목	내용
이유	API 키를 서버 환경변수에 두면 된다. 앱과 같은 TypeScript라 언어가 하나이고 요청·응답 타입을 공유할 수 있어 11주차 중간 발표까지 빠르게 만들 수 있다. 대기 시간 대부분이 Gemini·DB 응답이라 Node의 비동기 처리에 맞고, 서버가 상시 실행이라 콜드 스타트와 실행 시간 제한이 없다. 관계형 DB라 '같은 조합 1개', '글마다 반응 1개' 규칙을 DB 제약으로 보장한다. pgvector로 연관 단어 검색까지 같은 DB에서 처리한다. 익명 토큰은 서버가 직접 발급·검증해 '로그인 없음' 원칙을 지키면서 기기를 구분한다. Spring Boot도 가능하지만 앱과 언어가 달라지고 메모리·시작 시간 부담이 커서 이번에는 제외했다
감수한 단점	Supabase가 해 주던 일(익명 토큰 발급·검증, 호출 제한, HTTPS·배포, DB 호스팅)을 직접 만들고 운영해야 한다. 무료 플랜은 유휴 시 서버가 절전하거나 DB가 멈춰 첫 응답이 느릴 수 있어 5초 목표를 9주차에 미리 측정해야 한다. 앱을 재설치하면 이전 '내 글'과 내 반응이 내 것으로 연결되지 않는다
결정한 사람 (A)	지태웅 (팀 확인 필요)

결정 기록 2

항목	내용
결정 제목	같은 조합은 DB에서 먼저 찾고, 없을 때만 LLM을 부른다 (조합 키에 방향성 포함)
상황	세 단어 조합마다 LLM을 부르면 토큰이 낭비되고 매번 5초를 맞추기도 어렵다. 목표 4번은 '저장된 조합을 다시 요청하면 AI 호출 0회'이다. 방향성이 프롬프트에 들어가면 같은 단어 3개여도 결과가 달라야 한다
선택지	① 매번 LLM 호출 ② DB를 먼저 조회하고 없을 때만 LLM 호출·저장 ③ 모든 조합을 미리 생성해 두기 (카테고리당 단어 100개면 조합 161,700개라 현실적이지 않음)
결정	②. 조합 키 = 카테고리 + 정렬한 단어 3개 ID(단어 순서 무시는 제안, 팀 확정 필요) + 방향성(공백 정리). 키에 유일 제약을 걸어 같은 조합이 동시에 들어와도 한 건만 저장한다. LLM 실패 결과는 저장하지 않는다

항목	내용
이유	목표 4번(AI 호출 0회)을 직접 충족하고, 저장된 조합은 DB 조회만 하므로 응답도 빠르다. 방향성을 키에 넣어야 사용자가 정한 방향과 다른 결과가 재사용되는 일을 막는다. 자체 AI 모델 개발은 이미 Won't로 정했다
감수한 단점	같은 조합이면 항상 같은 결과라 새로움이 떨어진다(위험 4, '새로 생성' 허용 여부 미정). 방향성은 자유 입력이라 키 종류가 늘어 재사용 비율이 낮아진다(방향성 없이 뽑은 조합이 주로 재사용된다). 부적절한 결과가 저장되면 다른 사용자에게 반복 제공되므로 신고·숨김 장치가 필요하다. 조합이 처음인 사용자는 항상 최대 5초를 기다린다
결정한 사람 (A)	지태웅 (팀 확인 필요)

두 안을 맞춘 내용과 팀 확인 필요

기술 선택은 팀 상황을 더 잘 반영한 안(Supabase 중심)을 바탕으로 하되, 서버는 Node.js(Supabase 제외), 클라이언트는 React 기반, LLM과 임베딩은 Gemini로 바꿨습니다. 제 안에서는 구성도 그림과 공개 범위 해석, 신고·새로 생성 정리를 가져왔습니다.

항목	통합 결정	근거
클라이언트	React Native (Expo) + TypeScript (Flutter는 대안으로 남김)	요청에 따라 React 기반으로 변경. 서버도 TypeScript라 언어가 하나로 맞음. React 경험자 확인 필요
서버	Node.js (TypeScript) API 서버 (Spring Boot는 대안)	앱과 같은 TypeScript라 언어가 하나이고, 상시 실행이라 콜드 스타트·실행 시간 제한이 없음. 대신 인증·배포·호출 제한을 직접 만들어야 함
인증	서버가 직접 발급하는 익명 토큰(JWT)	서버가 서명한 토큰이라 앱이 만든 UUID보다 호출 제한·반응 조작 방지에 유리. 발급·검증 코드는 직접 작성
조합 키	방향성 포함	방향성이 프롬프트에 들어가므로 키에 빠지면 다른 방향의 결과가 재사용됨
공유 공개 범위	'링크 복사'와 '피드에 올리기' 두 버튼이 공개 범위 선택을 대신함	기획서는 '범위를 사용자가 선택', 공유 시트 명세는 '범위 설정 화면 없음'이라는 충돌을 화면 추가 없이 해소
데이터	서버 9덩어리 + 기기 1덩어리	익명 사용자·만드는 법·연관 점수·신고를 별도로 두고, 공유 글에 공개 범위를 추가

항목	통합 결정	근거
API	핵심 9개 + 추가 5개, 경로는 `/posts` 계열로 통일, 반응 취소는 `PUT` 하나로, 만드는 법은 AI 생성이 있어 `POST`	`POST /ideas`가 DB에서 꺼냈는지 여부를 돌려줘 목표 4번을 확인할 수 있게 함
배포	Google Play 내부 테스트 + 서버는 PaaS, DB는 관리형 PostgreSQL	실기기 시연에 충분. 리전은 가입 때 확인(가까울수록 지연이 작음)
LLM 모델	Gemini API Flash 계열, 버전은 연동 때 확인	요청에 따라 Gemini로 변경. 빠르고 저렴한 소형 모델이 5초 목표에 맞음. 임베딩도 Gemini로 맞춰 제공사를 하나로 줄임

다음은 이 문서만으로는 정할 수 없어 팀이 확인해야 하는 항목입니다.

항목	내용
팀명	이 문서는 5조로 고쳐 쓰였고, 기획서는 4조로 적혀 있음
공유 피드와 3주차 문서	팀이 정리한 내용에 따르면 3주차 문서의 '안 만들 것' 중 커뮤니티·SNS 피드와 공유 피드(07)가 충돌함. 3주차 원문은 확인하지 못했음
React 환경과 담당자	React(Native)·TypeScript·Node.js·SQL을 다룰 수 있는 팀원, 주차별 담당자 배정
새로 생성 허용 여부	허용하면 조합 키 유일 조건을 '조합 키 + 버전'으로 바꾸고, 어느 결과를 보여줄지 정해야 함
단어 순서 무시	조합 키에서 단어 순서를 무시하는 것은 가정이며 기획서의 결정 요청 항목임
외부 서비스 확정	Gemini 모델 버전(LLM·임베딩), 무료 사용량·호출 한도·데이터 사용 조건, API 비용, 서버 호스팅(PaaS)과 pgvector를 지원하는 관리형 PostgreSQL의 서비스·리전·무료 한도
5초 목표 측정	무료 PaaS의 절전 후 첫 응답과 Gemini 호출 시간을 9주차에 미리 측정
신고 화면	신고 기능은 위험 대응에 있지만 화면 명세가 없음. 화면을 만들지, 관리자가 DB에서 직접 숨길지 결정